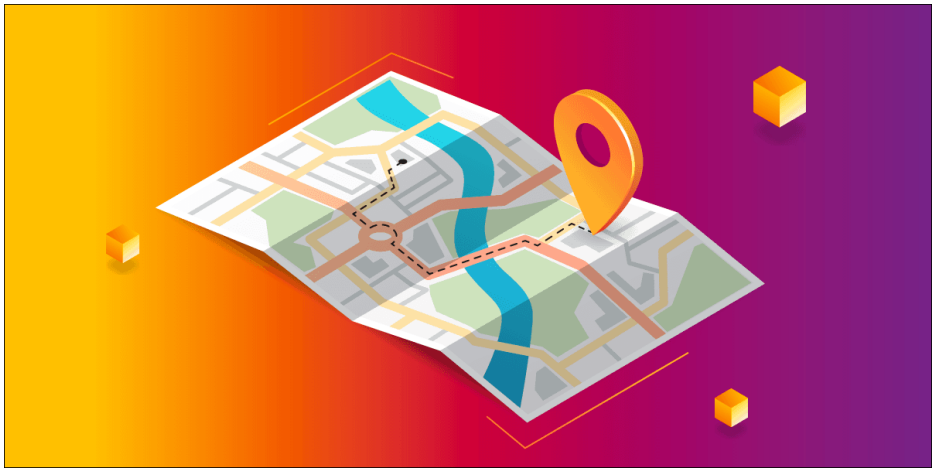
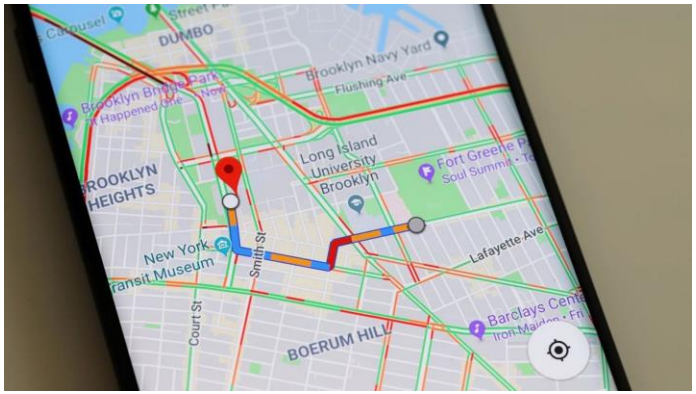


# Breadth First Search.
# Introduction to Shortest Path Algorithms.

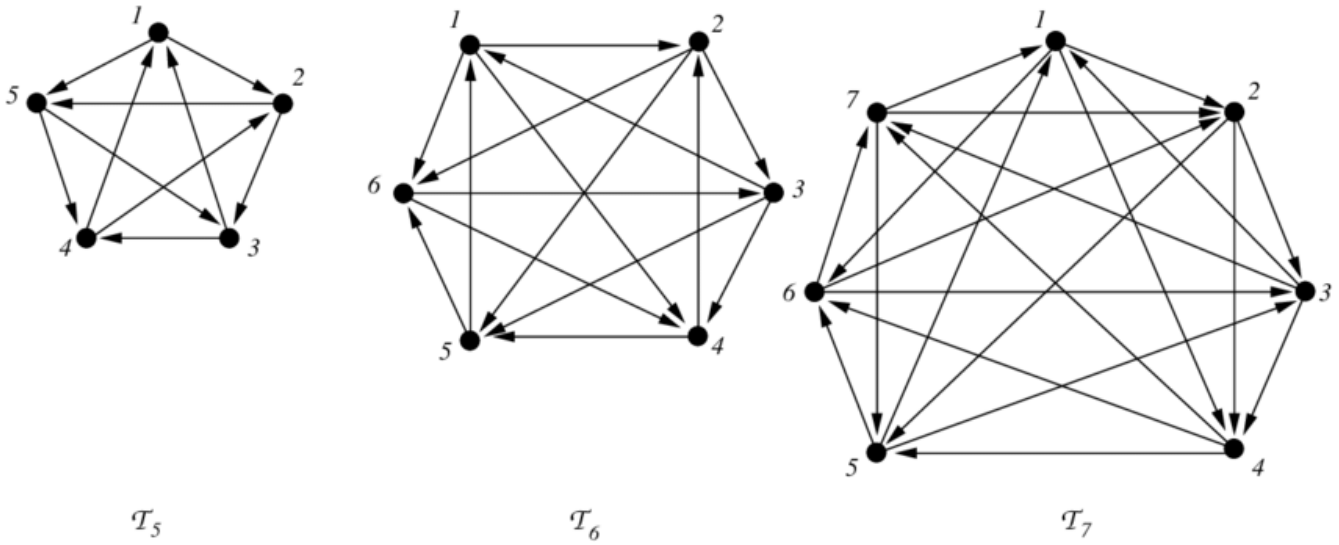


## Review - Lecture 3

Hamilton path (cycle) visits every node exactly once.
= A simple path (or cycle) of length n-1 (or n)

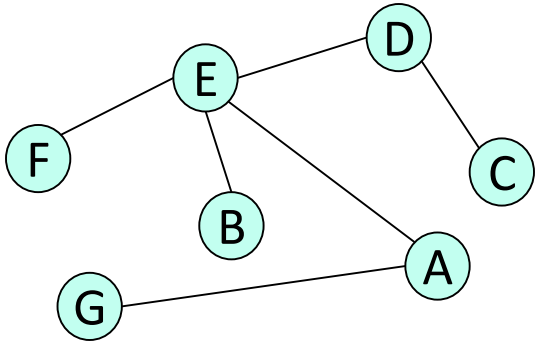
- No efficient (poly-time) algorithm is known.
- Some sufficient conditions:
  - Undirected graphs: Dirac, Ore
  - Directed graphs: Tournament



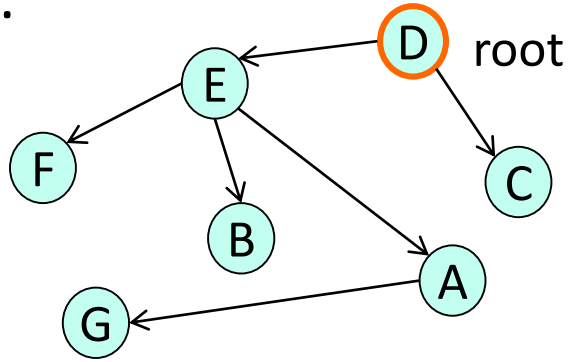

$T_5$ $T_6$ $T_7$

## Review - Lecture 3

- An undirected graph is a tree if it is connected and contains no cycles.




- A directed graph is a tree if it has a root and its underlying undirected graph is a tree.

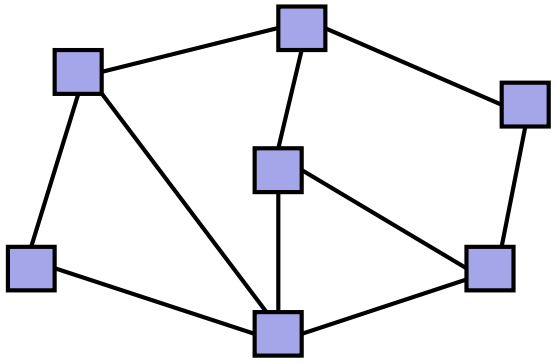



We proved several equivalent conditions

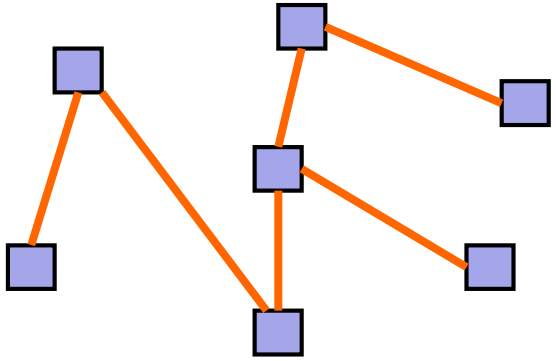
## Review - Lecture 3

Spanning tree:

Input: An undirected connected graph G = (V,E).



Output: E’⊆ E such that (V,E’) is a tree



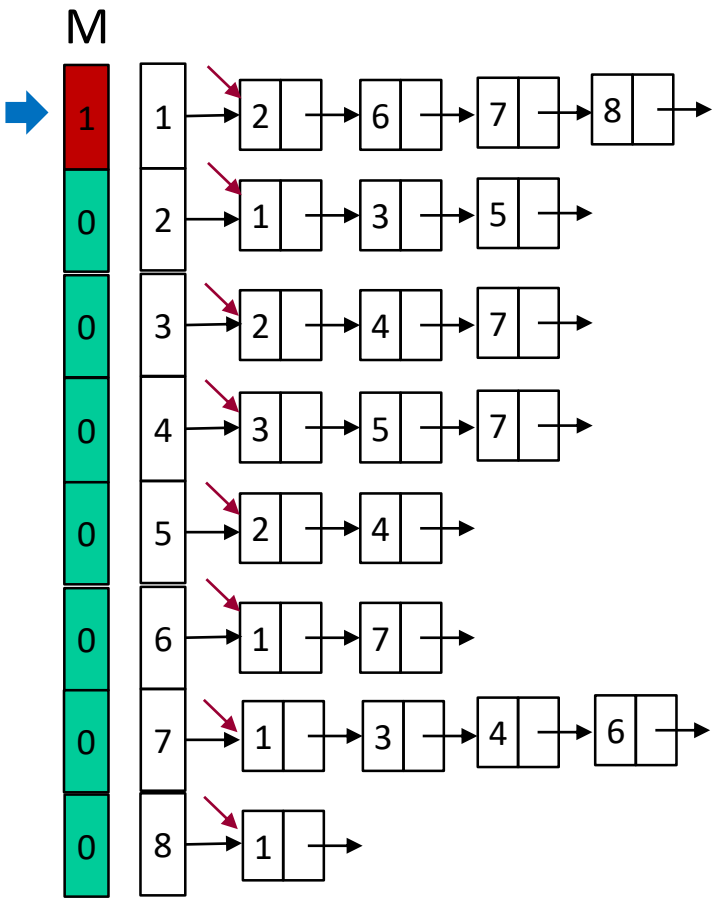
# Review - Lecture 3

DFS Algorithm:

Recursive marking algorithm

Initially every vertex is unmarked.

```
DFS(i: vertex)
   mark i;
   for each j adjacent to i do
      if j is unmarked then DFS(j)
end{DFS}
```



## Review - Lecture 3

Use DFS to get a spanning tree:

Init E'=∅

Whenever a node u detects an unmarked neighbor v, the edge (u,v) is added to E'.

We proved:

The graph (V,E') is connected and cycles-free.

Remark: For an unconnected graph, by running DFS on every component, we get a spanning forest of G.

## Review - Lecture 3

DFS analysis – implementation with adj. lists.

Time: Initialize Mark array: $O(|V|)$

During the execution, every node considers all its neighbors. Each neighbor in $O(1)$.

All together : sum degrees = $O(|E|)$.

Another view: Every edge is considered twice. Once from each endpoint. $O(1)$ per consideration.

Total : $O(|V|+|E|)$

Space: $O(|V|+|E|)$ for the adj. lists.

$O(|V|)$ for the recursion stack.

In a connected graph, $|E| \geq |V|-1$.

Therefore $|E|+|V| \leq 2|E|+1$ ➔ $O(|V|+|E|)=O(|E|)$.

## Review - Lecture 3

DFS – So far, we have used it to compute a spanning tree.

There are additional applications of DFS – anything that require to tour the whole graph.

In particular: DFS can be used to check connectivity (how?)

A more `popular' application: Maze solving.

In class: DFS in undirected graphs.
In the recitations (this week): DFS for directed graphs.

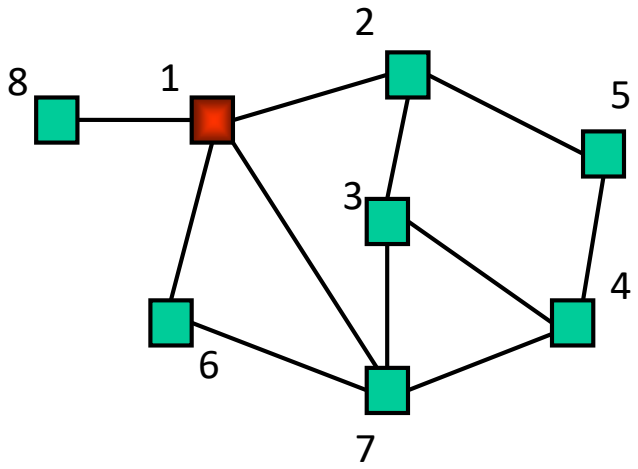
- Some applications of DFS for directed graphs:
  - Computing strongly connected components
  - Topological sort

# Spanning Tree using Breadth First Search (BFS)

```
Initialize E'=∅;
Initialize Q to be empty;
Enqueue(Q, s) and mark s;
while (Q is not empty) do
    i ←Dequeue(Q);
    for each j adjacent to i do
        if j is not marked then
            add (i,j) to E';
            mark j;
            Enqueue(Q, j);
```

s- an arbitrary node (to be the root)

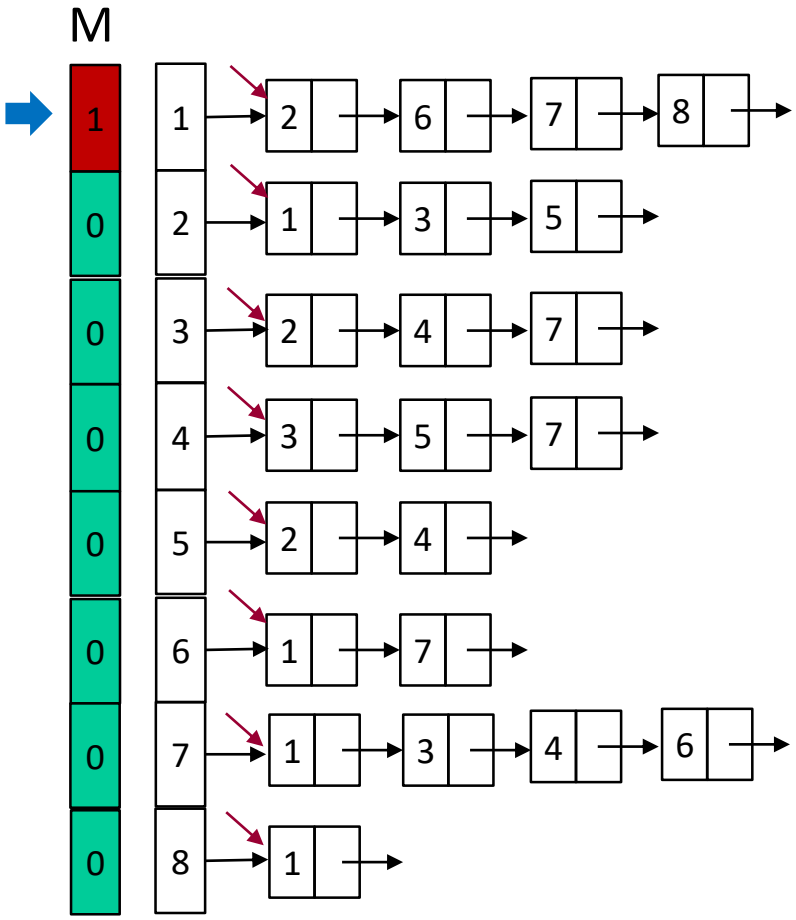
# BFS Example – with adj. lists



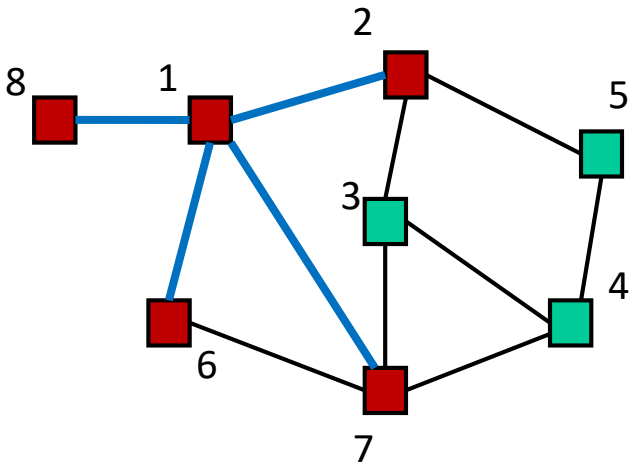

BFS(1)



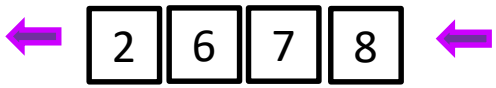

queue



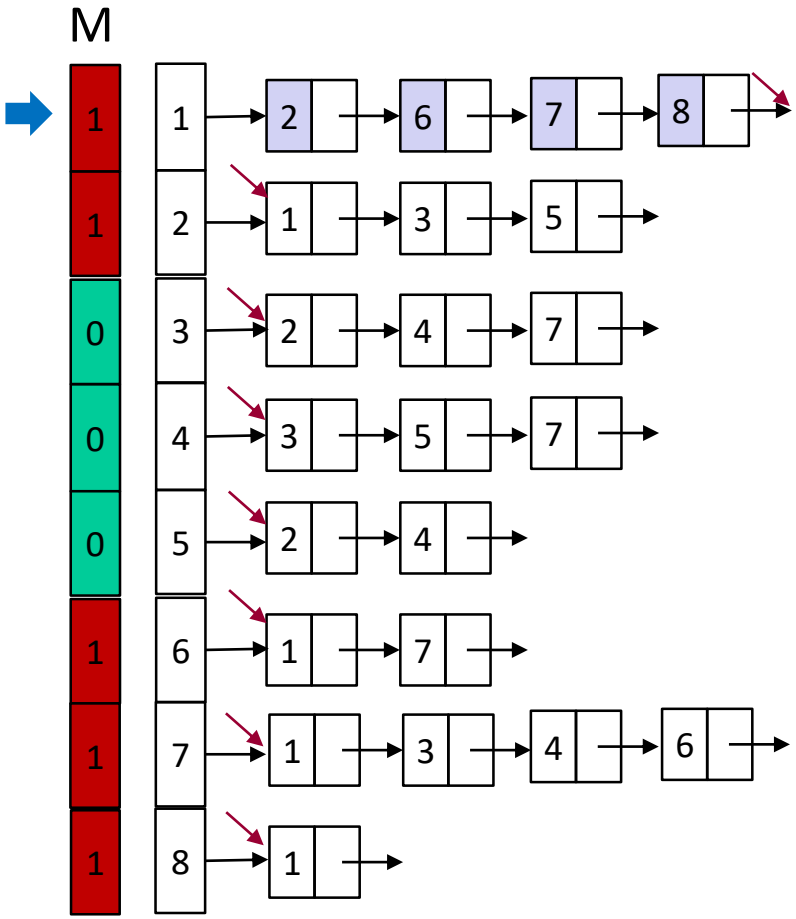
# BFS Example – with adj. lists



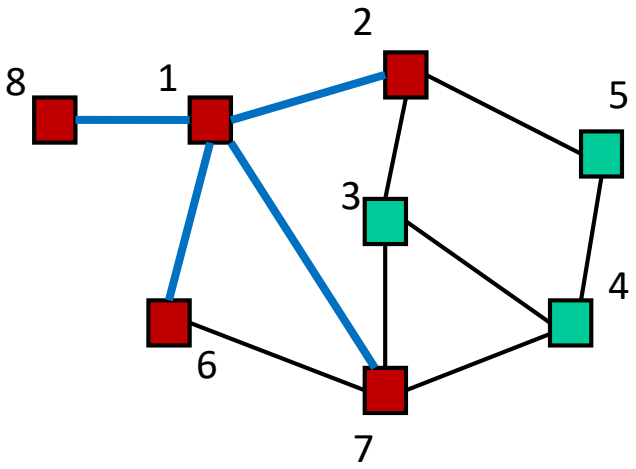

BFS(1)



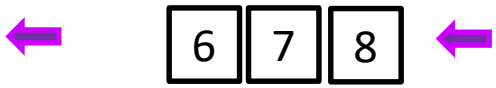

queue



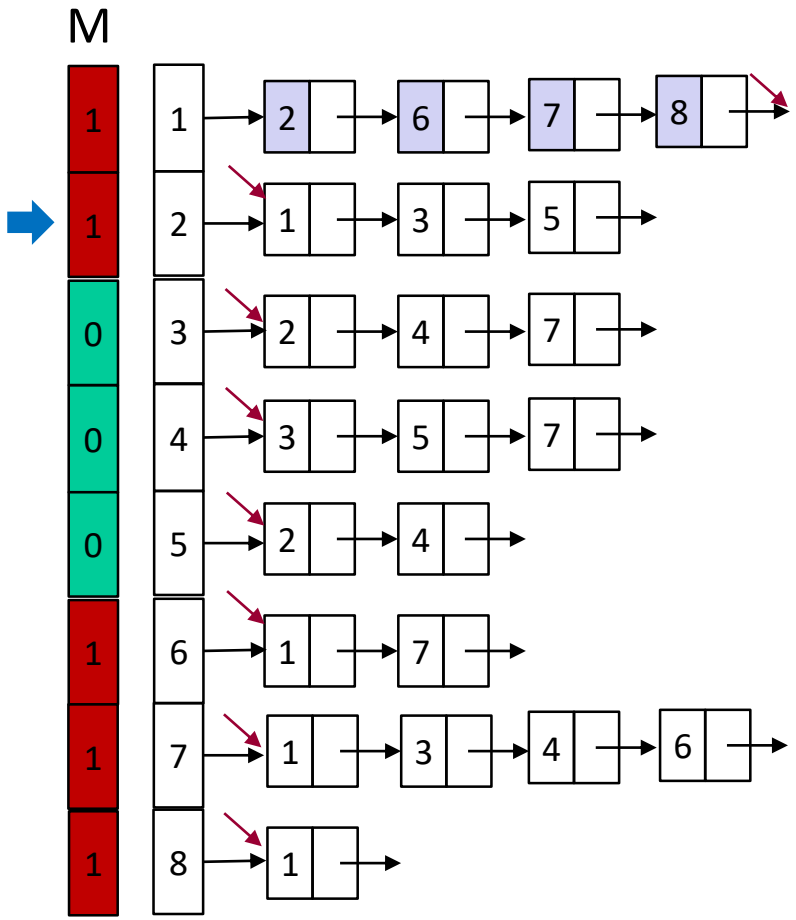
# BFS Example – with adj. lists



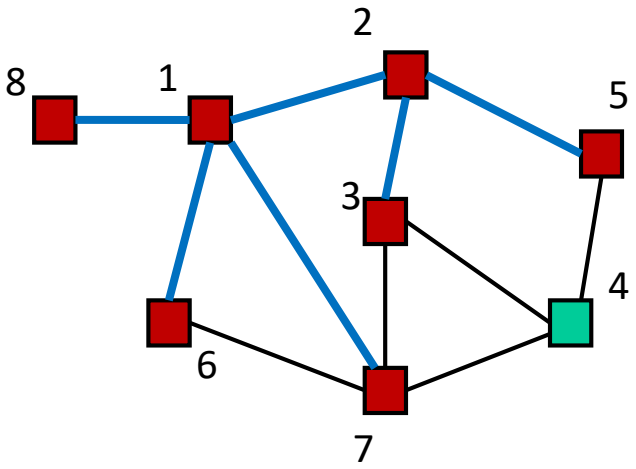

BFS(2)



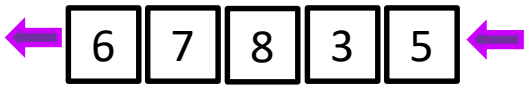

queue



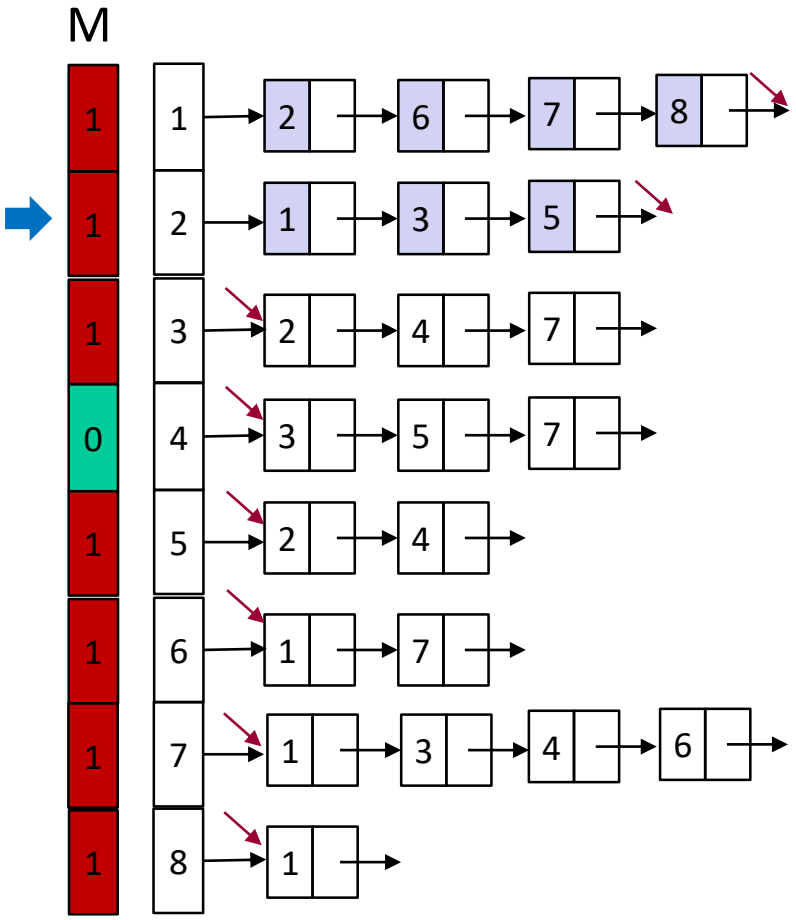
# BFS Example – with adj. lists



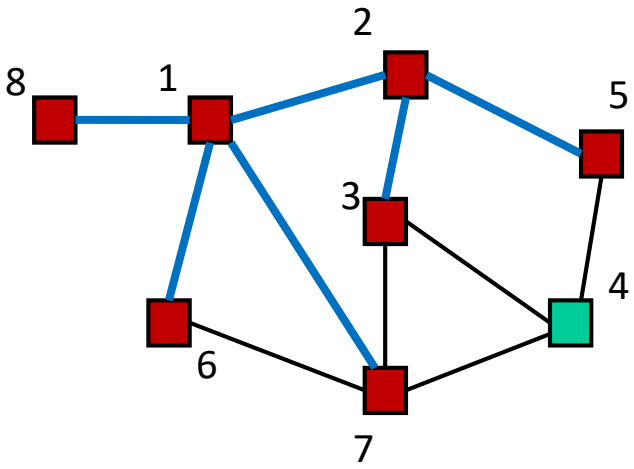

BFS(2)




queue



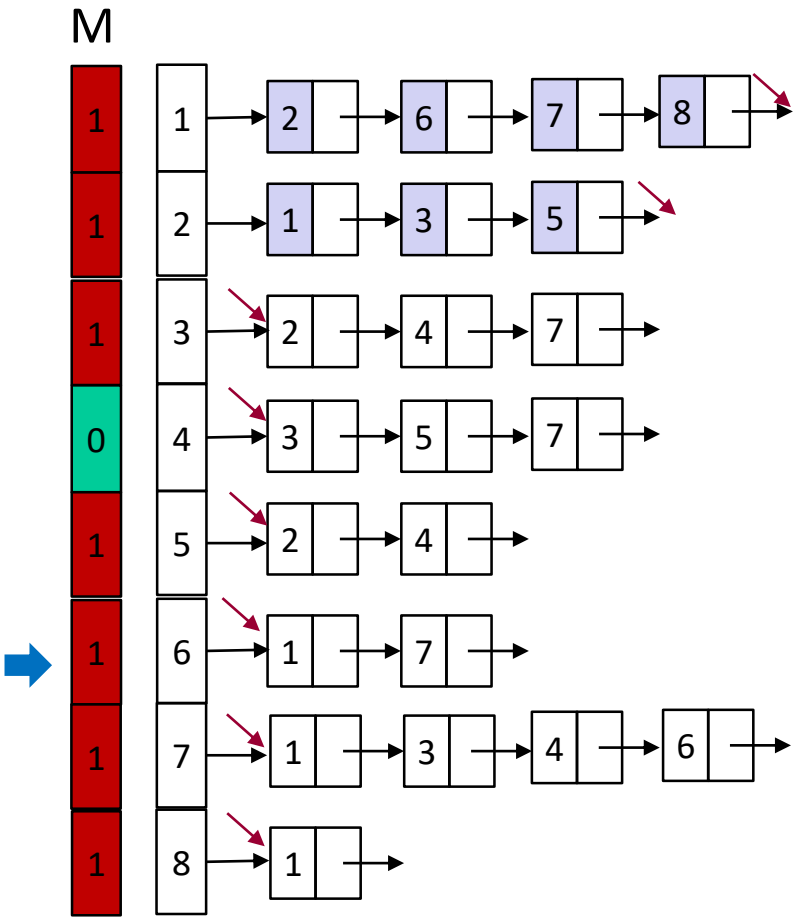
# BFS Example – with adj. lists



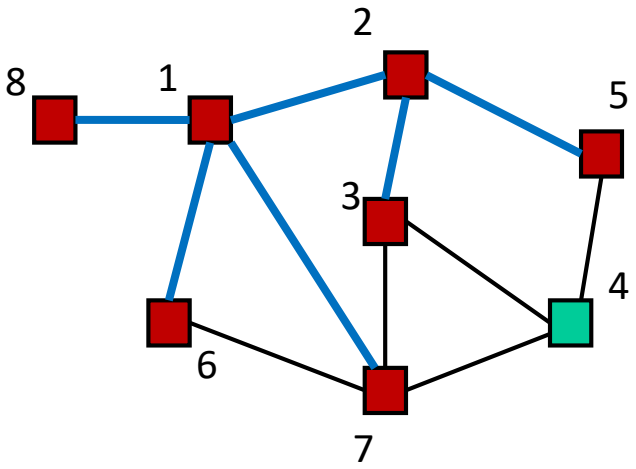

BFS(6)




queue



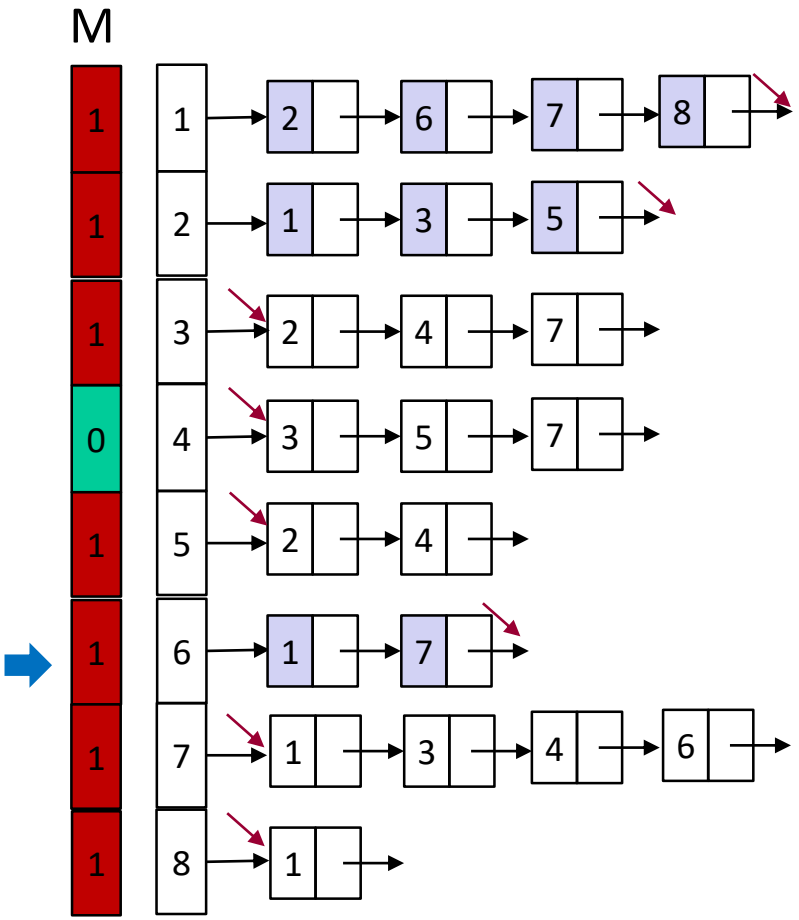
# BFS Example – with adj. lists



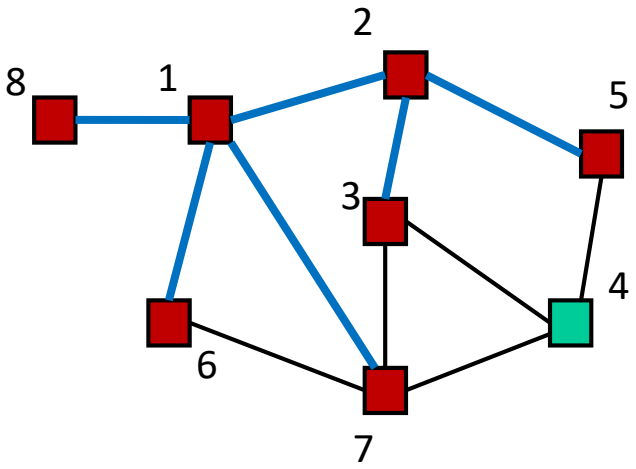

BFS(6)



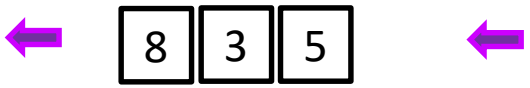

queue



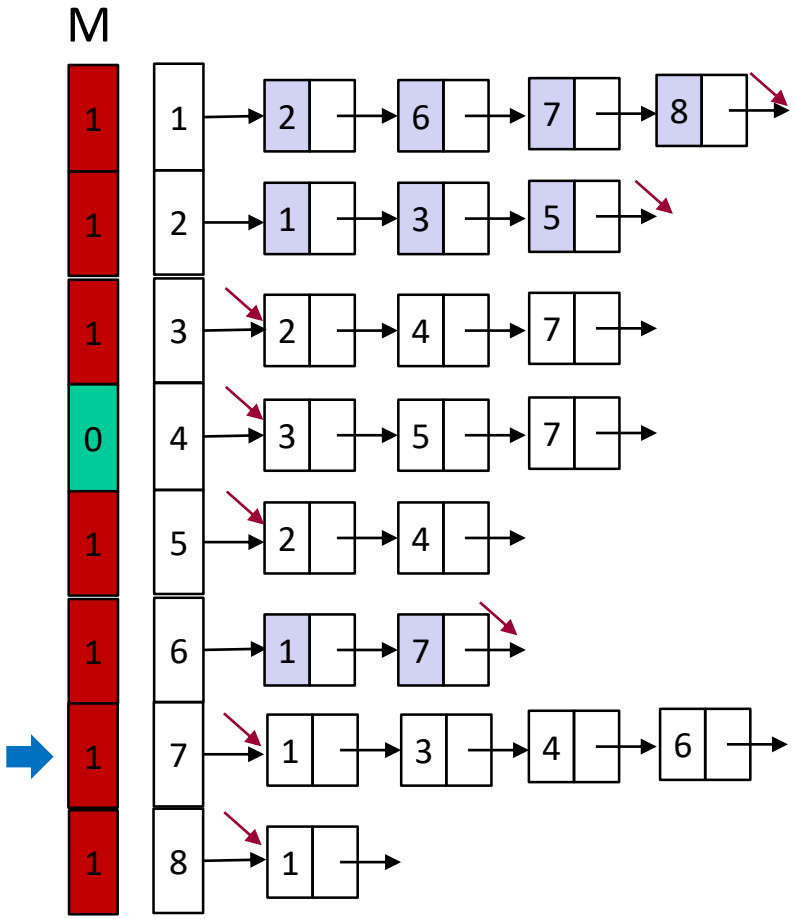
# BFS Example – with adj. lists



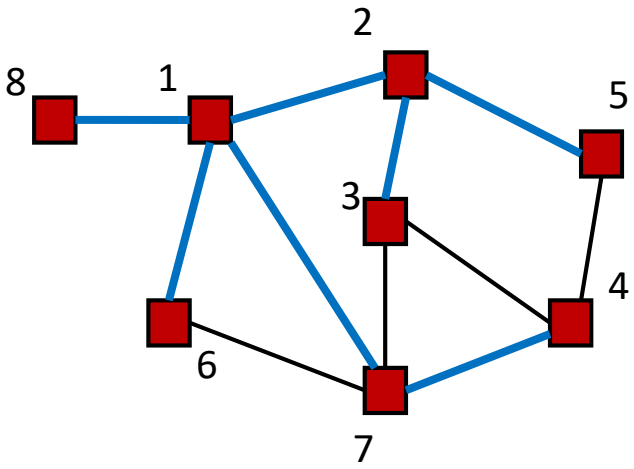

BFS(7)



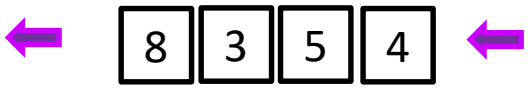

queue



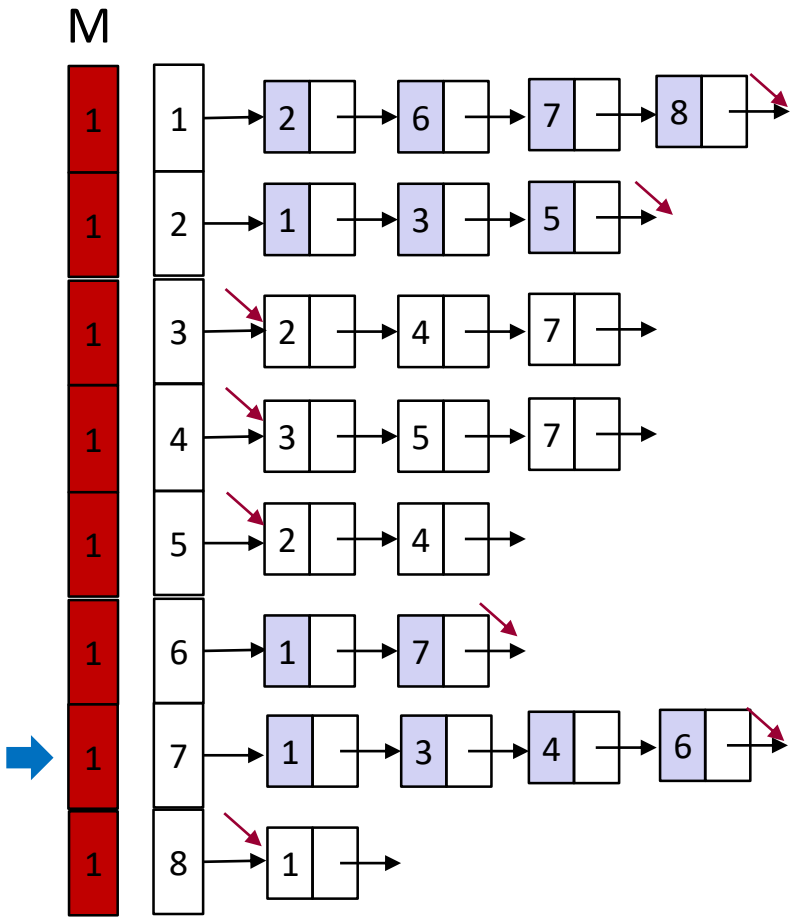
# BFS Example – with adj. lists



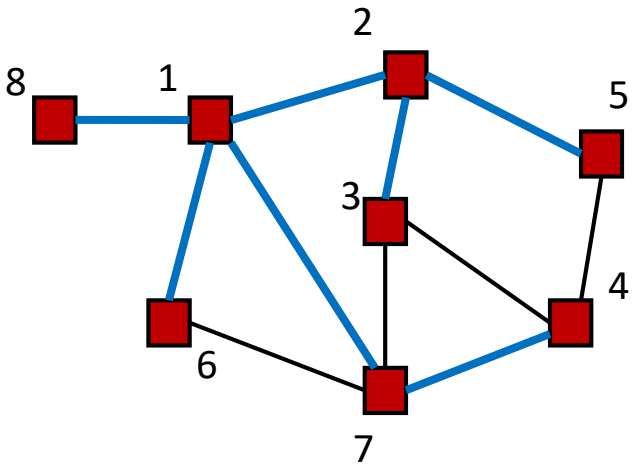

BFS(7)



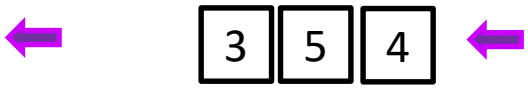

queue



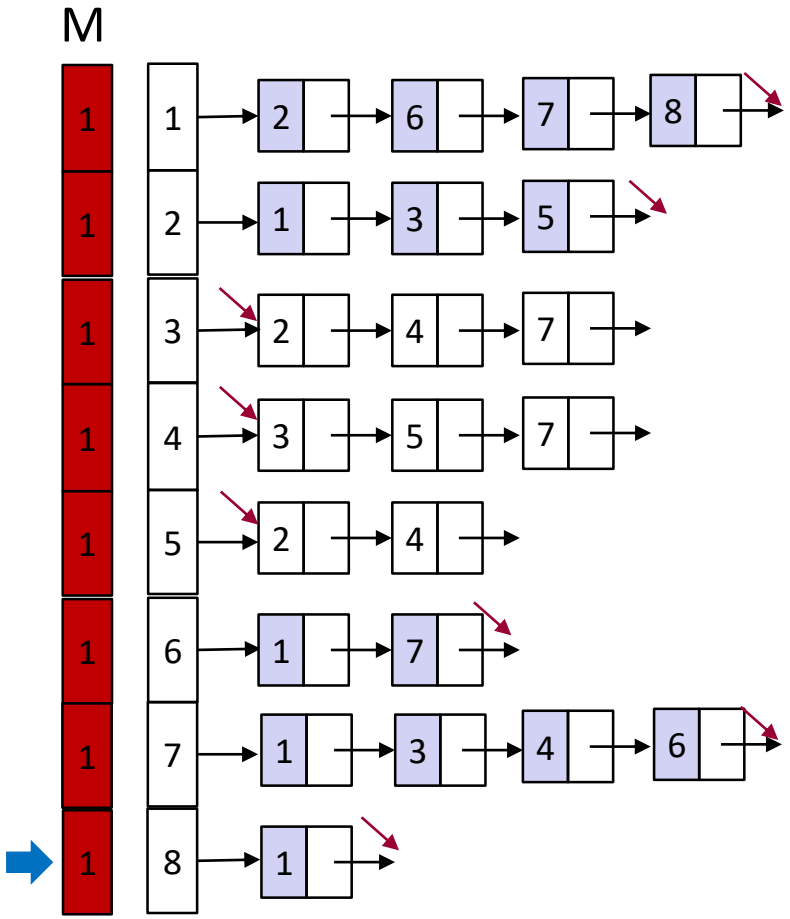
# BFS Example – with adj. lists



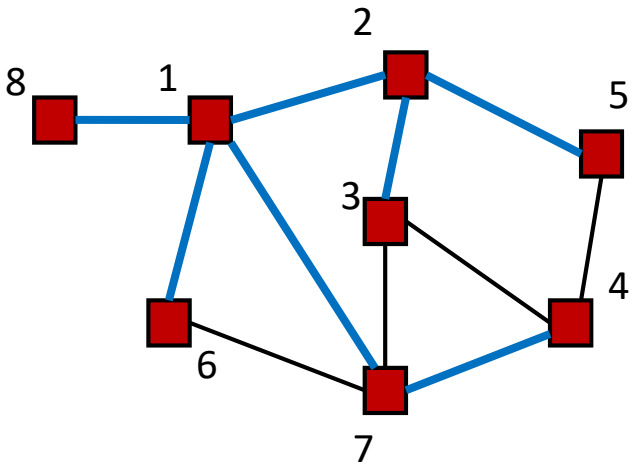

BFS(8)



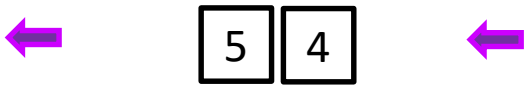

queue



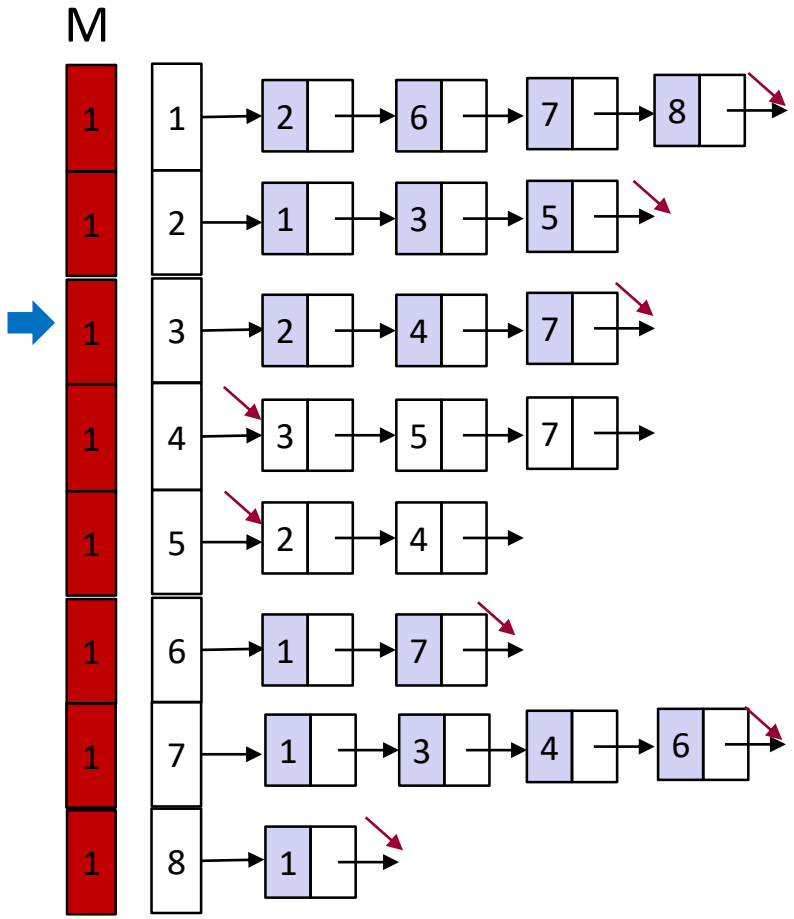
# BFS Example – with adj. lists



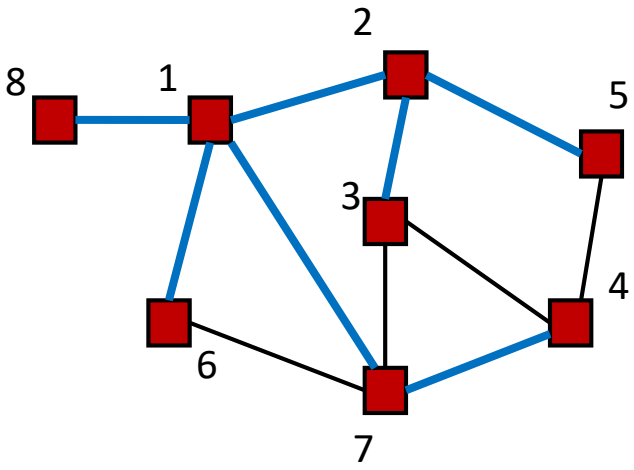

BFS(3)




queue



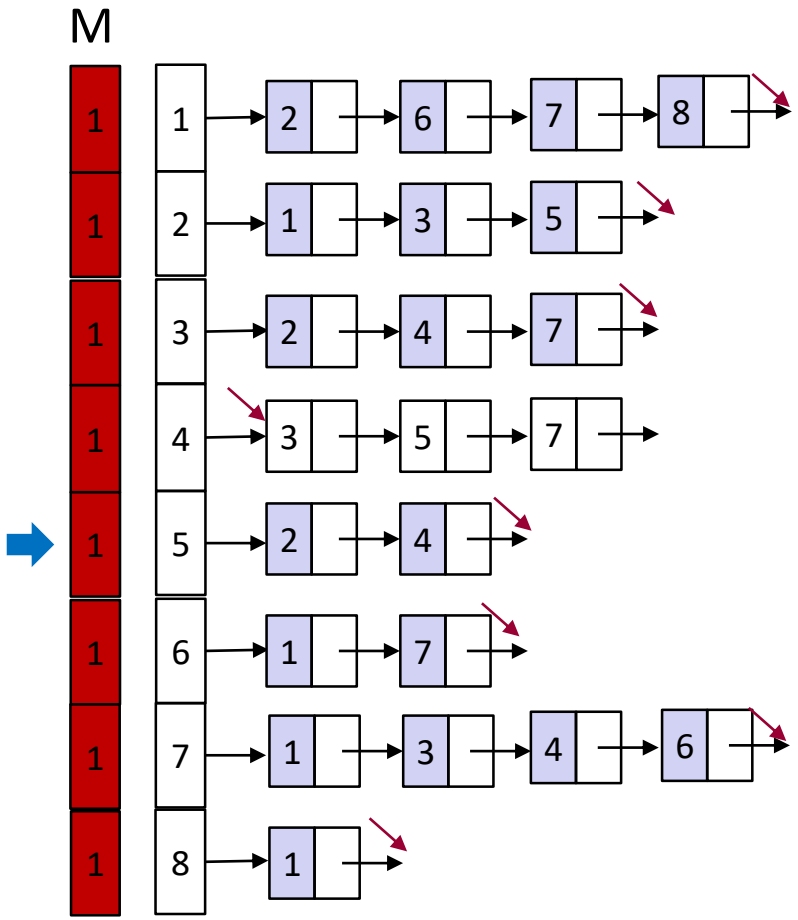
# BFS Example – with adj. lists



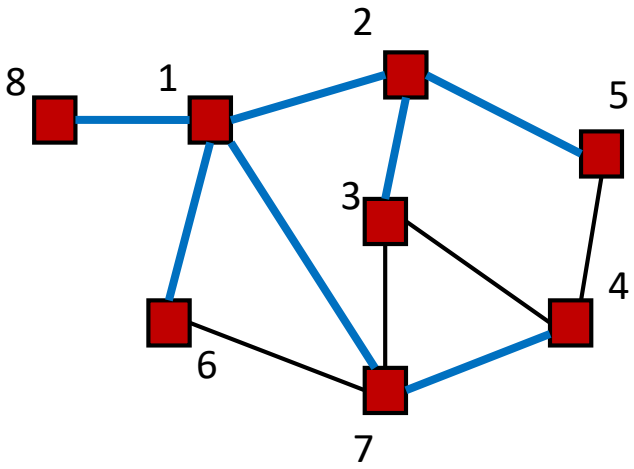

BFS(5)




queue

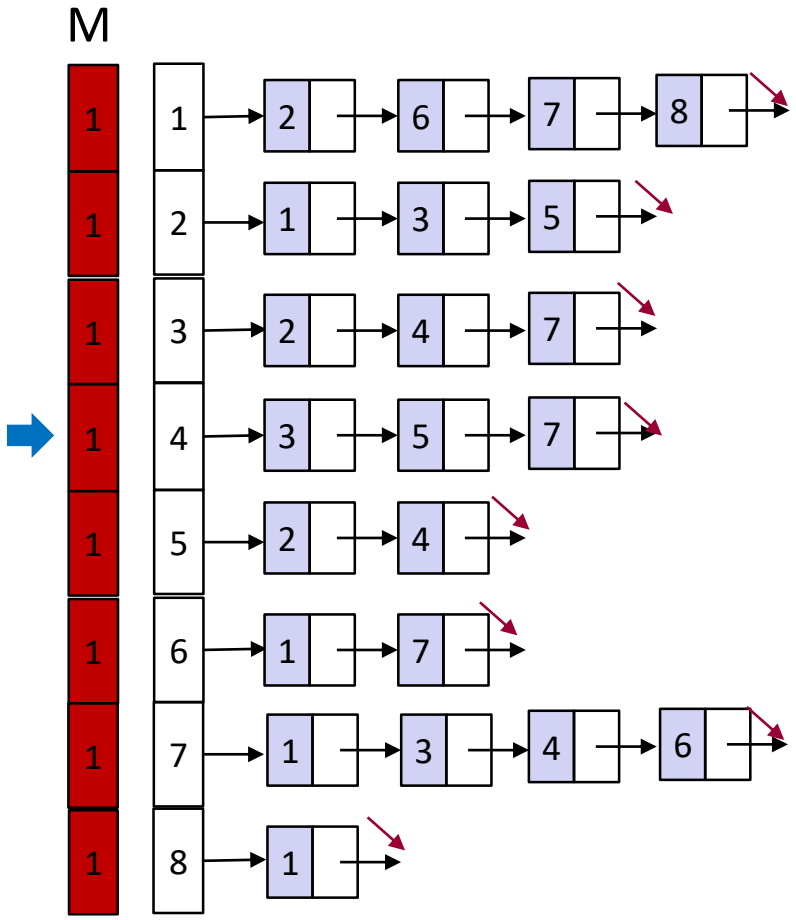


# BFS Example – with adj. lists

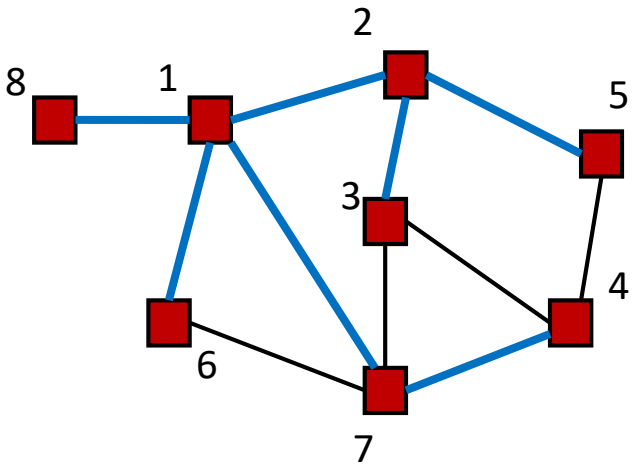


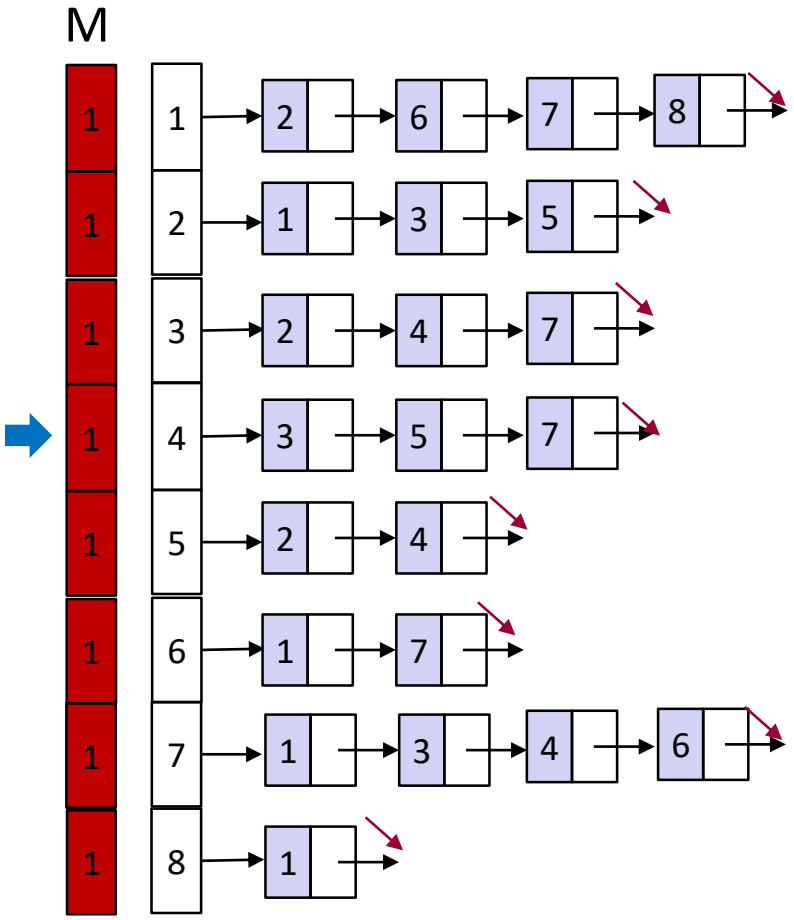

BFS(4)

queue

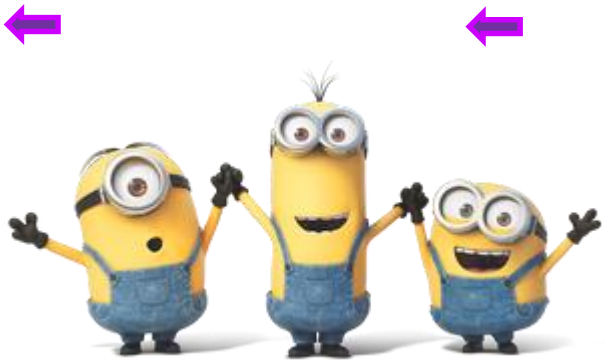


# BFS Example – with adj. lists




Empty Queue – Done!



# Shortest-path Algorithms

- Scenario: One router creates messages (source). Each message needs to reach other routers (one or more) along the shortest possible path.
- Abstraction: given a vertex s, find the shortest path from s to any other vertex of G.
- Path's length = number of edges in the path.
- Other shortest path problems (stay tuned…):
  - Different edges have different weights (length, delay, cost, etc.)
  - All-pair shortest path problem: no specific source.
- For a node v, the length of a shortest path from s to v is denoted d(s,v). [the distance from s to v].

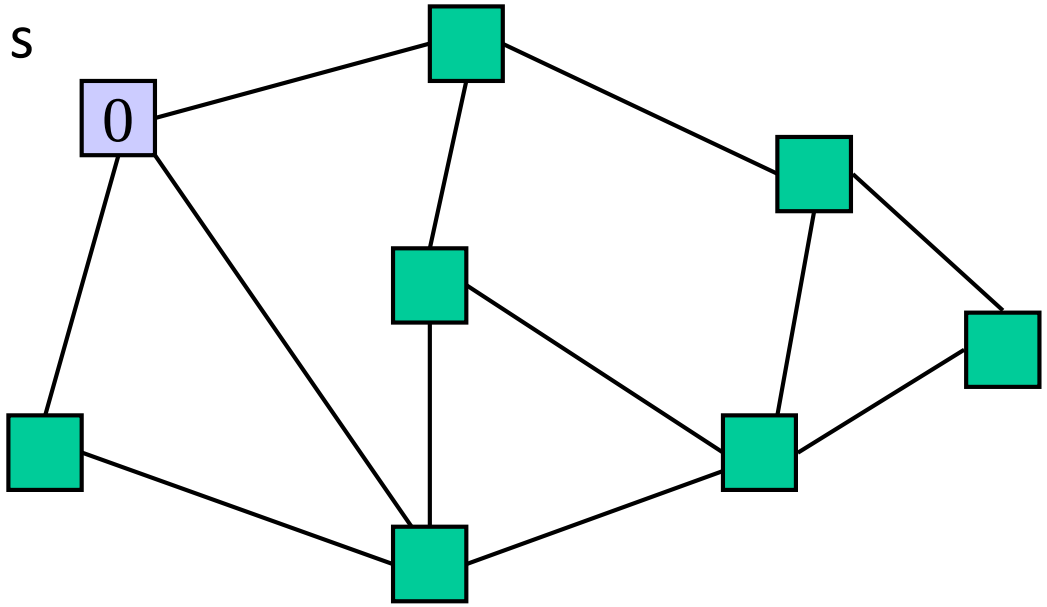
# Using BFS for Shortest-path

- Given a vertex s, find the shortest path from s to any other vertex of G.

A 'centralized' version of BFS:

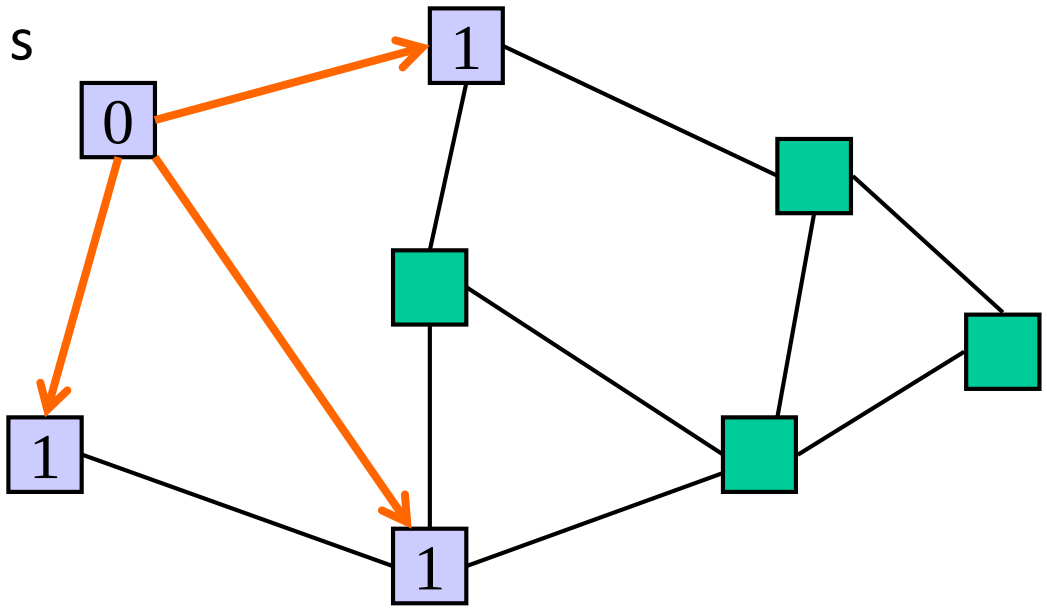
1. Label vertex s with 0.
2. $i \leftarrow 0$
3. Find all unlabeled vertices adjacent to at least one vertex labeled i. If none are found, stop.
4. Label all the vertices found in (3) with i + 1.
5. $i \leftarrow i + 1$ and go to (3).

# BFS for Shortest Path (i=0)



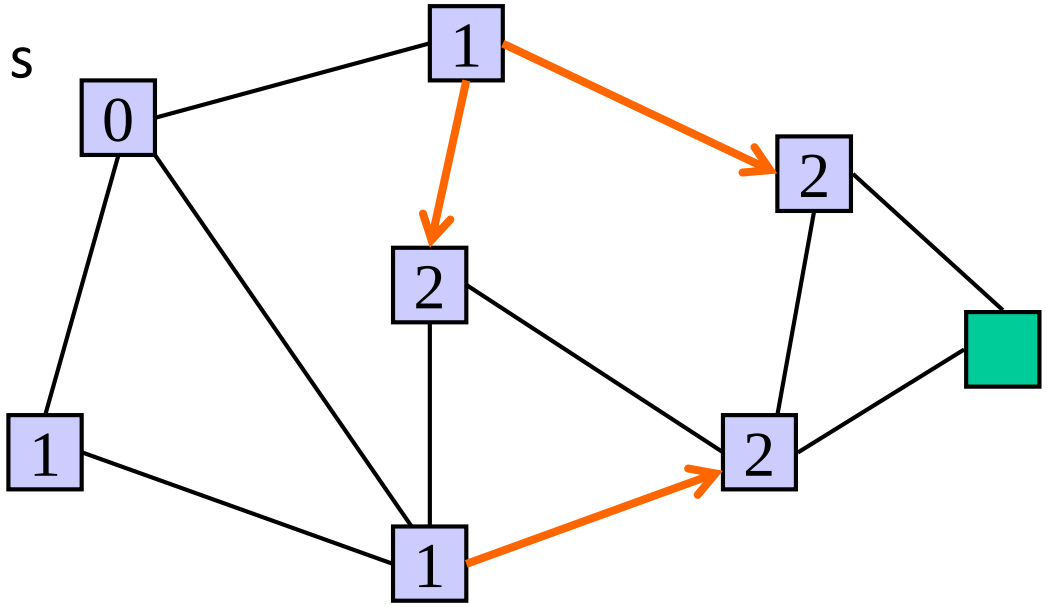

Vertices whose distance from s is 0 are labeled.

# BFS for Shortest Path (i=1)



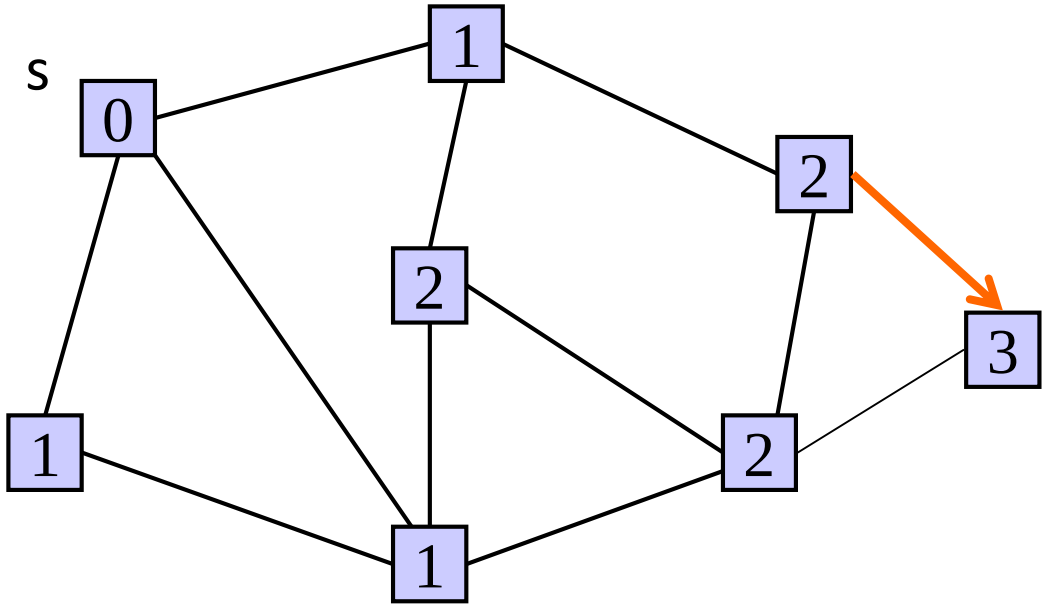

Vertices whose distance from s is 1 are labeled.

# BFS for Shortest Path (i=2)




Vertices whose distance from s is 2 are labeled.

# BFS for Shortest Path (i=3)




Vertices whose distance from s is 3 are labeled.

In the next iteration we find out that the whole graph is labeled and we stop.

# BFS for Shortest Path

Note: the 'centralized' version is for simplification only. When implemented, we need the queue as before.

```
Initialize E' to be empty;
Initialize Q to be empty;
Enqueue(Q, s) and label(s)=0;
while (Q is not empty) do
    i ←Dequeue(Q);
    for each j adjacent to i do
        if j is not labeled then
            add (i,j) to E';
            label(j) ←label(i) + 1;
            Enqueue(Q, j);
```

Addition to spanning tree algorithm

# BFS for Shortest Path – correctness proof.

Theorem: Each vertex is labeled by its distance from s.

Proof: We show by induction on k that label(v)=k iff d(s,v)=k.

Base case: k=0, the only node labeled 0 is s and indeed d(s,s)=0.

Step: Assume the claim is valid for all l<k and prove for k.

label(v)=k iff d(s,v)=k.

# BFS correctness proof.

$\Leftarrow$ Assume d(s,v)=k. We show that v would be labeled k. Let s-$v_1$-….-u-v be a shortest path from s to v. It must be that s-$v_1$-….-u is a shortest path from s to u, as otherwise, it is possible to shorten the shortest path to v.

By the induction hypothesis, label(u)=k-1. In the iteration in which u is active, it will consider v and label(v) will be determined to be k-1+1=k. This label will not change later.

It is not possible that v is labeled already by a label less than k, because, by the induction hypothesis, it means that d(s,v)<k.

label(v)=k iff d(s,v)=k.

# BFS correctness proof.

$\Rightarrow$ Assume that label(v)=k. We show that d(s,v)=k.

v is labeled k by some neighbor u for which label(u)=k-1. By the induction hypothesis, d(s,u)=k-1. Together with the edge (u,v) we have a path of length k from s to v. Thus, $d(s,v) \leq k$.

In addition, $d(s,v) \geq k$ since if it were less than k, then by the induction hypothesis, v was supposed to be marked already.



# BFS למציאת מסלולים קצרים - הוכחת נכונות

משפט: כל צומת מסומן במרחקו מ-s

הוכחה: נוכיח באינדוקציה על k ש- label(v)=k אמ"מ d(s,v)=k .

בסיס האינדוקציה: k=0 , הצומת היחיד המסומן 0 הוא s ואכן d(s,s)=0.

צעד: נניח שנכון לכל l<k ונוכיח לגבי k.

label(v)=k $\Leftrightarrow$ d(s,v)=k.

# הוכחת נכונות BFS

1. $\Leftarrow$ נניח שהמרחק מ-s ל-v הוא k (כלומר d(s,v)=k), נראה ש-v יסומן k.

יהי s-$v_1$-$v_2$-....-u-v מק"ב מ-s ל-v. בהכרח s-$v_1$-$v_2$-....-u הוא מק"ב מ-s ל-u, אחרת קיים מסלול קצר יותר אל v.

מהנחת האינדוקציה label(u)=k-1. כשהצומת u יהיה פעיל, הוא יתייחס ל-v, ואם אינו מסומן עדיין, יסמן את v ב-k-1+1=k. סימון זה לא ישתנה בהמשך הריצה. לא יתכן ש-v כבר מסומן בסימון נמוך מ-k, כי אז, לפי הנחת האינדוקציה, יש מסלול קצר מ-k בין s ל-v.

label(v)=k $\Leftrightarrow$ d(s,v)=k.

# הוכחת נכונות BFS

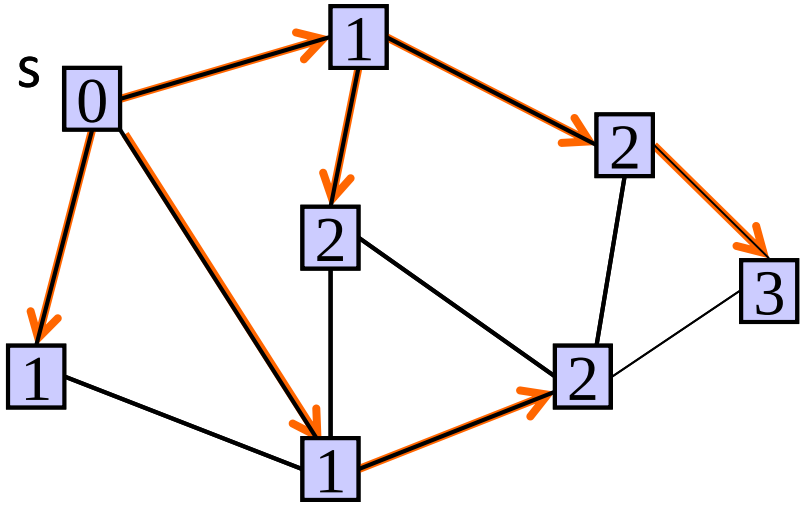
2. $\Rightarrow$ נניח ש-v מסומן k. נראה שהמרחק מ-s ל-v הוא k.

v מסומן ע"י איזשהו שכן u עבורו label(u)=k-1. מהנחת האינדוקציה, המרחק מ-s ל-u הוא k-1. יחד עם הקשת (u,v), מתקבל מסלול באורך k מ-s ל-v. לכן d(s,v)≤k.

בנוסף d(s,v)≥k כי אם המרחק היה קטן ממש מ-k, אז לפי הנחת האינדוקציה כבר היה מסומן, בסתירה לכך ש-u סימן אותו.

# The BFS Tree

For any $v \neq s$, let $p(v)$ be the vertex that 'discovered' $v$ in BFS.




$T=\{(p(v),v)\}$ is a directed spanning tree rooted in $s$, and for each vertex $v$, the path from $s$ to $v$ in $T$ is a shortest path from $s$ to $v$ in $G$.

*why?*

*s is a root.* $d_{in}(s) = 0$ and for every other vertex *v*, $d_{in}(v) = 1$.

# Shortest-path Algorithms

- Scenario: One router creates messages (source). Each message needs to reach other routers (one or more) along the shortest possible path.
- Abstraction: given a vertex s, find the shortest path from s to any other vertex of G.

- Using BFS, we solved the problem assuming all edges have unit length.
- A more practical setting: edges are not uniform.
- Tons of applications.

# Single-Source Shortest Paths

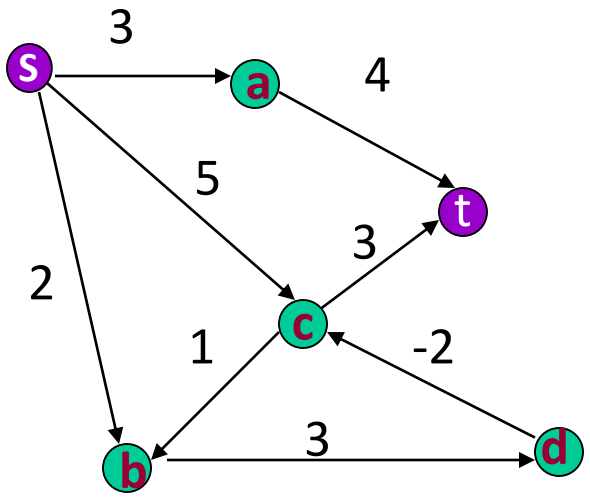
- Assume each edge (u,v) has non-negative weight w(u,v).
- The weight reflects the cost of using this edge. May correspond to distance (km) /cost ($) /delay (minutes) /resource consumption /risk / more.
- weight of a path = total weights of all edges on path.

**Input:** A graph $G=(V,E)$, a weight function $w:E \rightarrow R$, a source vertex $s \in V$.

**Output:** For each vertex $v \in V$, $d(s,v)$ – the length of a shortest path from $s$ to $v$ (= v's distance from s).

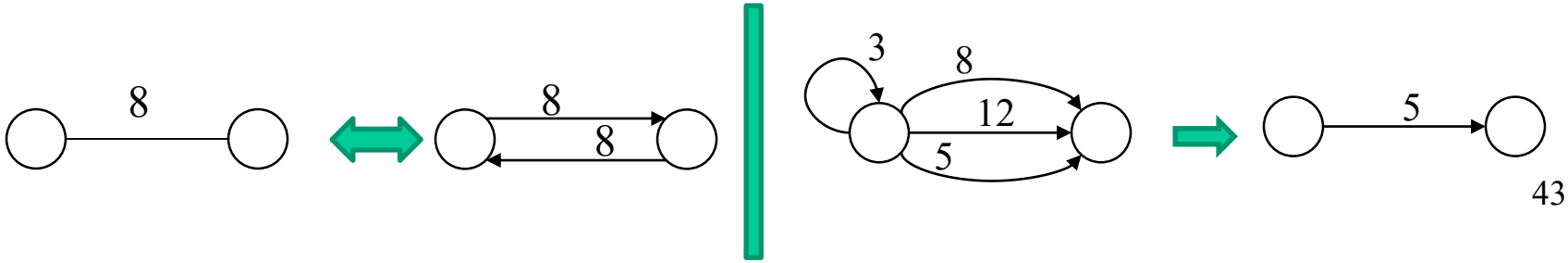
# Weighted Directed Graphs

- The distance from s to t is the length of the shortest path (according to weight) connecting them.
- Denoted by d(s,t).
- Example: The distance from s to t is 6. The shortest path is s-b-d-c-t.
- There are other paths from s to t whose weights are 7 (s-a-t) and 8 (s-c-t).

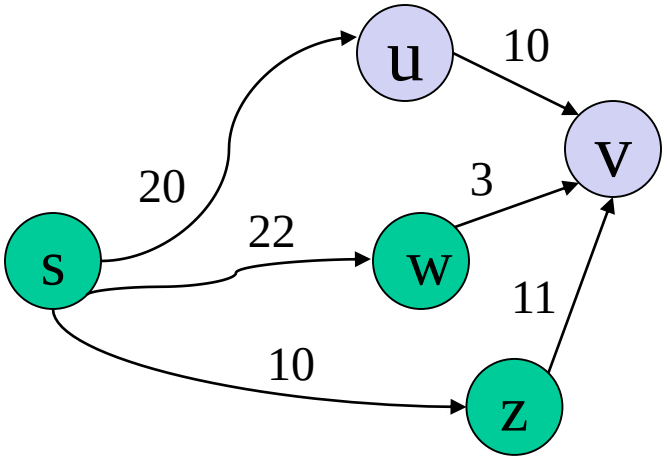


# Single-Source Shortest Paths

- We will assume that the input graph is directed.
- All our algorithms can be adopted to undirected graphs – by replacing each edge e=(u,v) by two anti-parallel edges (u,v) and (v,u) both having weight w(e). The time complexity will remain the same.
- We also assume that the input graph is simple:
  - If there are parallel edges, all but the cheapest one can be removed.
  - If there is a self-loop, it can be removed (or if it is negative, no solution exists).



# Properties of Distances




d(s,v) ≤ 20+10
d(s,v) ≤ 22+3
d(s,v) = 10+11

Fact 1:

For every edge u,v:

$d(s,v) \leq d(s,u) + w(u,v)$

Fact 2:

For every v (except s) there exists a vertex u such that

$d(s,v) = d(s,u) + w(u,v)$

# The General Method

Init:

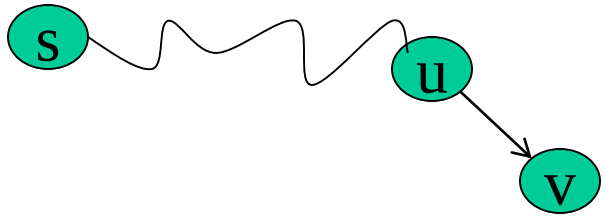
$s.d \leftarrow 0$

$s.\pi \leftarrow$ null

for all other v:

$v.d \leftarrow$ infinity

$v.\pi \leftarrow$ null




Main loop:

repeat

choose an edge (u,v)

relax(u,v)

**relax(u,v):** **(u,v)∈E**

if v.d > u.d + w(u,v) :

$v.d \leftarrow u.d + w(u,v)$

$v.\pi \leftarrow u$

v.d – length of shortest path detected so far

$v.\pi$ - predecessor of v (parent).

# Properties of this Method

Any algorithm of this type satisfies:

1. For each v, the value v.d is monotonically decreasing during the execution of the algorithm.
2. For each v, and any time during the run of the algorithm $v.d \geq d(s,v)$.

3. Let u be the last vertex on a shortest path from s to v. If at some point $u.d = d(s,u)$ is true, and then relax(u,v) is performed, then from this time on, $v.d = d(s,v)$ holds.

# v. $\pi$ – used to retrieve a shortest path

path(v)

if v≠s :

    path(v.$\pi$)

print v

Observation:

Once for all v∈V, it holds that v.d = d(s,v), the procedure path(v) prints a shortest path from s to v.

